

JENKINS

Continuous Integration & Continuous Delivery Reference Guide

1. Continuous Integration & Continuous Delivery (CI/CD)

In modern software engineering, Continuous Integration (CI) and Continuous Delivery/Deployment (CD) form the backbone of automated release pipelines. CI focuses on the automated integration, building, and testing of code changes committed to a shared repository, enabling rapid detection of integration issues. CD extends this pipeline by automating the deployment of tested applications to staging and production environments, reducing manual intervention and cycle time. Jenkins serves as the industry-standard automation engine to orchestrate these operations.

2. Jenkins Controller-Agent Architecture

Jenkins is built for horizontal scalability using a distributed model composed of a Controller (host) and multiple Agents (workers):

- **The Controller (Primary Node)**

The Controller acts as the orchestrator. It manages user configurations, access controls, plugin lifecycles, job scheduling, and hosts the web user interface. Running heavy build executors directly on the Controller is an administrative anti-pattern, as build failures or resource exhaustion can destabilize the entire server.

- **Agents (Executor Nodes)**

Agents are thin JVM processes running on external systems (virtual machines, bare-metal servers, or containers). The Controller delegates compilation, testing, and deployment scripts to these agents. This separation ensures resource isolation, allows building across different operating systems, and facilitates horizontal scalability.

3. Jenkins Pipelines (Pipeline-as-Code)

Jenkins Pipelines enable developers to define release processes using code, typically saved as a text file named "Jenkinsfile" at the root of the project source control repository. Standardizing pipelines as code ensures that build steps are version-controlled, easily auditable, and reviewed along with the application source code.

Declarative vs. Scripted Syntax

Jenkins supports two pipeline programming formats:

1. Declarative Pipeline: A structured model that uses a predefined schema. It is easy to write, read, and maintain. It incorporates validation mechanisms and clear stage blocks. This is the recommended choice for modern pipelines.
2. Scripted Pipeline: A procedural model based on Groovy scripting. It provides absolute control and flexibility over the flow but requires complex scripting, making it harder to maintain and troubleshoot.

Example Declarative Jenkinsfile

```
pipeline {
  agent any // Allocation of any available execution agent
  stages {
    stage('Build') {
      steps {
        echo 'Building codebase...'
        sh 'npm run build'
      }
    }
    stage('Test') {
      steps {
        echo 'Running unit tests...'
        sh 'npm test'
      }
    }
  }
}
```

4. Core Strengths & Plugin Integration

The massive adoption of Jenkins is driven by its open-source nature and extensibility. Jenkins offers over 1,800 community-contributed plugins, enabling direct integration with git repositories, build runners, container ecosystems (Docker, Kubernetes), artifact storage systems, cloud providers, and notifications (Slack, Webhooks).

5. Jenkins Administration Best Practices

- **Maintain Pipeline-as-Code**

Always define pipeline workflows in a Jenkinsfile. Avoid visual configuring inside the Jenkins UI.

- **Zero Executors on Controller**

Set controller executors to 0 to prevent building jobs locally. Always route builds to external agents.

- **Implement Credential Masking**

Use Jenkins Credentials Store for storing API keys, passwords, and SSH keys. Never hardcode secrets.

- **Enforce Build Discarders**

Regularly discard old build history and artifacts to prevent filling up the JENKINS_HOME disk storage.

- **Automated Backups**

Schedule periodic backups of the JENKINS_HOME configuration directory to allow fast disaster recovery.

Conclusion

While newer SaaS CI/CD tools have emerged, Jenkins remains highly relevant due to its unmatched customizability. Adhering to administrative best practices helps maintain a stable, scalable, and secure automation server for enterprise DevOps software pipelines.